Anchorium Autonomous Trading Bot Project Report

Implementation and validation explained in plain language

Prepared for Adesh and Parth of Team Anchorium

Project status as of 2 October 2026

The crypto infrastructure is implemented around the preserved VWAP strategy. The project can read the competition markets, evaluate candidates, simulate and account for trades, recover local state, export evidence and replay one complete captured scan.

The latest saved test suite contains 678 passing tests. Current operation remains PAPER with real exchange orders disabled. The recorded public runs took no trades; wallet readiness and profitable competition performance have not yet been verified.

This report explains the inherited project, every migration stage, the current operating flow, implementation files, observed validation, limitations and the remaining practice milestones. A glossary and a fictional trade example help explain the technical terms and accounting.

How to read this report

Sections 1 to 7 establish the context. Sections 8 to 25 explain how each component works. Sections 26 to 30 show the evidence, remaining work, operating commands and code map. Sections 31 to 33 provide the glossary, worked example and source references.

Contents

1 Project overview and current position....3
2 What the project is trying to achieve....3
3 The original bot and inherited improvements....4
4 The migration rules and why they matter....4
5 Migration phases zero through five....5
6 The competition venue and market universe....6
7 How the current system is organized....7
8 The strategy explained in plain language....7
9 Candles and the continuous trading clock....8
10 How the scanner selects markets....9
11 Market specifications and order precision....10
12 Prices that must remain separate....10
13 Local order execution and partial fills....11
14 Position management and exits....11
15 Shared USDPR accounting....12
16 Fees spread slippage and funding....12
17 Position sizing and risk controls....13
18 Feed freshness and risk states....14
19 Reconnection and paper recovery....15
20 Read only exchange account truth....15
21 Event timing and complete candidate traces....16
22 Results files and session identity....16
23 Paper metrics and competition scoring....17
24 Replay and the blocked live mode....18
25 Wallet readiness and secret protection....18
26 Testing and the observed public results....19
27 What is implemented and what remains....20
28 The controlled practice sequence....21
29 Operating the current project....21
30 Code and file reference....22
31 Glossary for the team....24
32 Example of one complete paper trade....25
33 Evidence and further reading....26

1 Project overview and current position

Team Anchorium has converted the surrounding infrastructure of its Indian-market trading bot into a Hyperliquid testnet project for Perpetual Alpha. The existing VWAP strategy remains unchanged. The new software can discover the competition markets, read public prices and candles, evaluate the strategy, reject unsuitable opportunities, simulate orders, maintain a local account, record its decisions and replay a captured session.

The project currently operates in PAPER mode. A paper trade is a local simulation: it changes the bot's own records but does not place an order on an exchange. REPLAY is also available for a complete recorded single-scan session. The LIVE interface exists as a blocked structure; signed trading, account approval and funded-wallet integration have not been enabled.

The latest saved automated test run contains 678 passing tests, with zero failures, errors or skipped tests. The strategy freeze check and all 12 Phase 0 safety checks were also verified again while preparing this report. The saved public audit session discovered ten markets and took no trades because every market had stale completed candles. Offline replay reproduced that session's diagnostics, decisions and portfolio exactly. These observations establish specific engineering behavior; they do not establish profitable trading performance.

The current migration branch is `perpetual-alpha-migration`. The protected baseline is the tag `nse-working-baseline`, pointing to commit `ed0dba5b0577da38a500d9dc208fe9938d3f3a14`. The repository's current commit at report preparation is `d6d6090c1d2066b3828105bdf0412e9f1d548d32`. Saved sessions also keep their own historical commit and source hashes, so an older result can be traced to the code that produced it.

Area	Current position
Strategy	Preserved and checked against the baseline
Public competition data	Implemented and observed in bounded public runs
Paper orders and accounting	Implemented and tested with controlled cases
Read-only actual account recovery	Implemented and fixture-tested; team address not verified
Replay	Implemented for a complete single-scan REST and clock tape
Signed exchange trading	Disabled; signed transport not implemented
Demonstrated profitable competition trading	No evidence yet

2 What the project is trying to achieve

The aim is an autonomous trading system that can make a complete, explainable decision without a person clicking every buy or sell. That requires more than a signal. It requires reliable market data, correct market identity, suitable order size, accurate costs, an account that reflects real fills, and a record showing why each action happened.

The competition uses perpetual futures. These are contracts that let a trader hold upward or downward price exposure without a fixed expiry date. The bot does not buy every underlying asset

directly. It trades contracts tracking those assets and must account for margin, leverage and funding. In this project, USDPR is the competition collateral and accounting unit.

The official evaluation includes final net return valued using oracle prices, daily equity-return volatility and a jury's assessment of a 2 to 4 page technical report. The jury considers methodology, AI/ML depth, originality, rigor, reproducibility and clarity. Entries must be algorithmic. The published ranking conditions also include trading on at least ten distinct days and reaching a minimum cumulative notional volume; the team must confirm the required volume with the organizers. This full project report can support the later, shorter submission. Official competition overview

The practical engineering objective is therefore to produce correct decisions and trustworthy evidence. Strategy optimization and new AI/ML methods remain separate future work. They have not been added during the exchange migration.

3 The original bot and inherited improvements

The original project traded Indian stocks through Angel One and SmartAPI. It used NSE stock names, instrument tokens, Indian trading sessions, whole-share quantities and Indian transaction-cost rules. Its strategy logic and its exchange plumbing were partly mixed together.

Several engineering improvements existed before the crypto migration. The inherited version 6 work introduced verified instrument identity, bid/ask-aware execution, cost-aware position sizing, a safety state machine, fill-based accounting, recovery checks and a durable SQLite journal. It also separated a capital-protection exit from the strategy's normal candle-close stop. That earlier checkpoint recorded 244 passing tests.

The stock-discovery work then expanded the ability to find eligible instruments. It added staged filtering, explicit fallback behavior and clearer selection reports. Subsequent discovery repairs recorded 303 and then 338 passing tests. These are historical checkpoints, not additional tests to add to today's total.

The next inherited repair addressed late startup, missing morning candles, partial first minutes, overlapping history, immature ATR readings, request pacing, cache validation, reserve watchlists and hidden rejection reasons. It recorded 382 passing tests. Its research tools kept experimental policies separate from production and reported inadequate data rather than creating artificial performance results.

That history matters because the migration inherited lessons about wrong instruments, optimistic fills, duplicated records and recovery. The new crypto path implements corresponding safeguards with perpetual-futures semantics. The inherited NSE-specific modules remain legacy references; their sessions, taxes, authentication and account assumptions are not the current competition body.

The repository also retains native C++ support in `libquant_engine`, including a tick parser, ring buffer, VWAP engine and Python bindings. These are inherited components. The current crypto paper runner uses its Python market adapters and frozen Python brain; this migration does not claim a new C++ performance benchmark or a native crypto execution path.

4 The migration rules and why they matter

The governing rule was to preserve the strategy and change its exchange and market infrastructure. VWAP conditions, LONG/SHORT decisions, EMA calculations, ATR calculations, stop/target

concepts and strategy-state behavior were frozen. A market-specific assumption could be adapted only as narrowly as needed and recorded clearly.

This makes comparisons meaningful. If the strategy and exchange plumbing changed together, the team could not easily tell which change caused a different result. Keeping the strategy constant makes data quality, market behavior, costs and execution easier to investigate.

Three settings define the development safety policy:

```
TRADING_VENUE=HYPERLIQUID_TESTNET
EXECUTION_MODE=PAPER
ALLOW_LIVE_ORDERS=false
```

The original NSE repository was not used as the migration work area. The cloned repository has its own Git identity, baseline tag and migration branch. Exchange writes are blocked inside execution methods, not only by a banner or a user-interface setting. An environment flag, a supplied wallet address or a successful public connection test cannot activate signed trading.

The strategy manifest checks one complete file, eight protected methods and 32 parameters. `signal_brain.py` remains byte-identical to the baseline. The eight-method protection extends to important management and observation functions in other files. A file hash is a content fingerprint: a changed byte changes the fingerprint and causes the freeze check to fail.

5 Migration phases zero through five

Phase 0 protected the working bot. The clone was identified as the Perpetual Alpha project, inherited broker execution was disabled and the paper/testnet policy was introduced. Angel One login entry points use a disabled client. Alternate order methods, protective orders and old activation evidence cannot bypass the lock.

Phase 1 froze the strategy. The baseline manifest, source checks and direct-input comparisons protect the existing brain. Rather than rewriting the indicator functions for crypto, the project converts exchange candles into the input format those functions already expect.

Phase 2 inventoried legacy dependencies. Searches classified references as REMOVE, REPLACE or KEEP AS LEGACY REFERENCE. The latest saved inventory contains 2,005 occurrences across 98 files: 1,788 retained references, 160 replacement dispositions and 57 removal dispositions. These counts include historical documents, tests and comments. They do not mean 2,005 active broker dependencies exist, or that every referenced legacy file has been deleted.

Phase 3 added generic exchange interfaces. Market data, orders, account information and reconciliation now have their own contracts. Strategy decisions do not need SmartAPI objects or Hyperliquid signing code. This separation allows the same strategy to consume paper or replay input safely.

Phase 4 connected public par market data. The client verifies the DEX and market universe before providing usable data. It supports REST reads and public WebSocket streams without a wallet private key. A missing, delisted or inconsistent required market aborts startup.

Phase 5 replaced hardcoded stock specifications. Runtime metadata supplies fractional size precision, maximum leverage, margin tiers and asset mapping. Decimal arithmetic and conservative normalization make the order quantity and limit price suitable for the selected market.

The first phases recorded 499 passing tests. Later policy-lockout work recorded 527, the integrated paper infrastructure 595, recovery and risk controls 645, and audit/replay work 678. Each later count is a complete suite at that checkpoint.

6 The competition venue and market universe

The current implementation selects Hyperliquid testnet and the HIP-3 DEX named `par`. HIP-3 allows a deployer to define a group of perpetual markets. Selecting the correct group matters because a request without the intended DEX can return another universe without an obvious error.

Market identifier	What the contract follows
par:BTC	Bitcoin
par:ETH	Ethereum
par:HYPE	HYPE
par:SP500	S&P 500 index
par:EWY	South Korea equity ETF reference
par:EWJ	Japan equity ETF reference
par:BRENT	Brent crude oil
par:GOLD	Gold
par:USDEUR	EUR/USD reference
par:USDJPY	USD/JPY reference

The public endpoints used by the project are:

```
REST reads    https://api.hyperliquid-testnet.xyz/info
Streaming     wss://api.hyperliquid-testnet.xyz/ws
```

The venue also has an `/exchange` endpoint for signed actions. The migration client does not implement a transport to that endpoint. It discovers the DEX index rather than treating a published number as permanent, and requires all ten expected markets. Newly listed active `par` markets can also enter the runtime universe. Official participant guide

7 How the current system is organized

The active flow begins with the public client and market adapters. Valid candles reach the scanner and frozen brain. A candidate then passes through risk checks, normalization and local execution. Confirmed local fills update the paper ledger and strategy manager. Reporting connects those stages into an auditable trace.

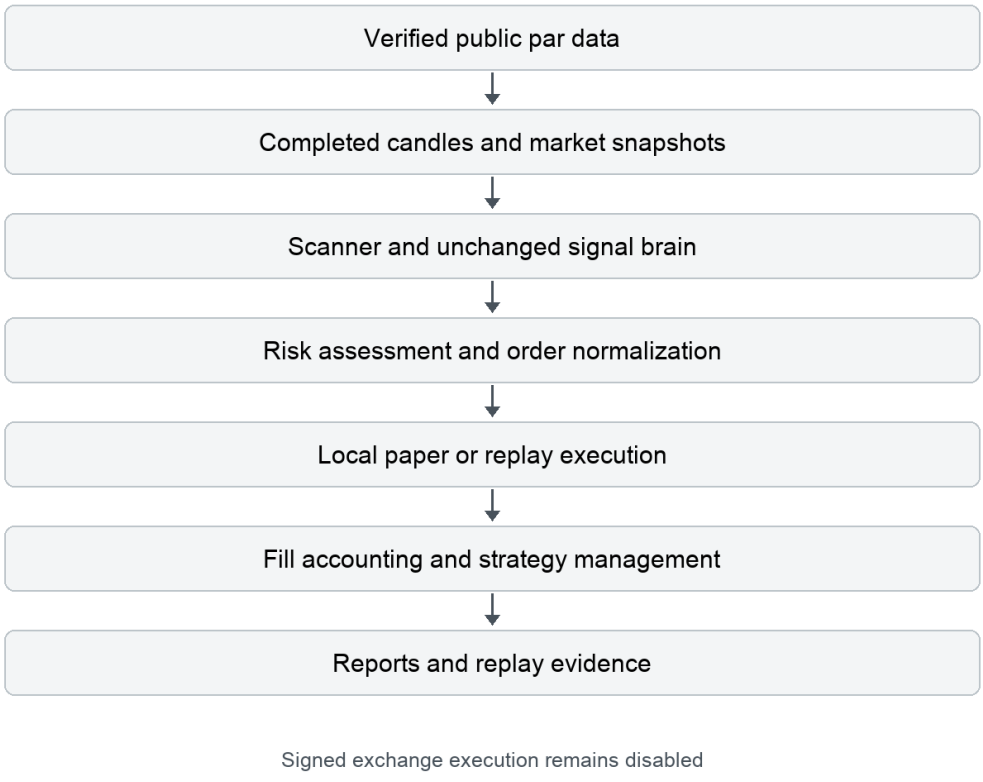


Figure 1 The current paper and replay operating flow

A separate read-only account observer can inspect a supplied public team address. Its exchange-reported positions, fees and funding remain separate from hypothetical paper cash. The LIVE executor is also separate and remains blocked.

Continuous mode uses separate public clients for the scanner and streaming worker. Those workers prepare observations. One main thread owns the local orders and ledger, which reduces the chance of two workers changing the same position or balance at once. The current user-facing operating tools are command-line runners and generated reports; a new web dashboard was not part of this migration.

8 The strategy explained in plain language

The strategy looks for a price relationship around VWAP while checking trend and volatility. VWAP is a running average price weighted by traded volume. A heavily traded bar contributes more than a lightly traded bar. In this brain, each bar's representative price is `(high + low + close) / 3`, and daily VWAP is the running sum of representative-price times volume divided by the running volume.

EMA means exponential moving average. It smooths closing prices while giving more weight to recent bars. EMA9 reacts faster than EMA21. Their relationship helps identify upward or downward momentum. Both retain their original periods and daily grouping behavior.

ATR means average true range. It measures typical price movement. True range uses the largest of the bar's high-low movement, high-to-previous-close movement and low-to-previous-close movement. The unchanged implementation uses a rolling arithmetic average across 14 bars, with the available early bars contributing during warmup. It does not replace this with a different smoothing formula during migration.

VWAP slope describes whether VWAP is rising or falling. The established lookback is ten bars, with the existing early-session calculation preserved. The volume average uses ten bars. These indicators support four setup types:

Setup	Plain-language idea
PULLBACK_LONG	Price was above VWAP, returns toward its allowed zone and closes above it again in an upward regime
BREAKOUT_LONG	Price closes decisively above its VWAP/ATR boundary with a volume surge in an upward regime
PULLBACK_SHORT	Price was below VWAP, rallies toward its allowed zone and closes below it again in a downward regime
BREAKDOWN_SHORT	Price closes decisively below its VWAP/ATR boundary with a volume surge in a downward regime

The current source, rather than its older introductory comments, defines the actual bands. The configured long and short pullback multipliers are 1.5 ATR. The breakout/breakdown close multiplier is 0.5 ATR. A volume-surge candidate uses 1.5 times the ten-bar volume average. Existing deep-breakdown and sharp-upward-spike filters remain in place.

A LONG candidate requires a positive VWAP slope and EMA9 at least as high as EMA21. A SHORT candidate requires a negative slope and EMA9 at most as high as EMA21. The brain waits for the established 15-bar session warmup and its own minimum ATR fraction of 0.0006, which is 0.06% of price.

The anti-spam lock prevents the same area from repeatedly producing adjacent entries. After a valid signal, the existing logic requires two consecutive bars displaced away from the relevant VWAP band before unlocking. A daily bias adds a separate direction preference using price versus VWAP, VWAP slope and EMA alignment. At least two of three votes establish LONG_BIAS or SHORT_BIAS; otherwise the result is NEUTRAL.

Those checks explain a signal. A signal still needs fresh data, appropriate selection, permitted state, positive economics and acceptable risk before it can become an order.

9 Candles and the continuous trading clock

A candle summarizes one minute's open, high, low, close and volume. The brain receives completed one-minute candles with aware UTC timestamps and per-bar base volume. The adapter validates the symbol, interval, timestamps, price relationships, numeric values and exact chronology.

An open minute is not treated as a completed bar. Duplicates, gaps, wrong identities, malformed boundaries and invalid prices are rejected. Missing minutes are not filled with invented prices or volume. The raw normalized candles are retained with session evidence so the team can examine what the brain actually saw.

The new session adapter uses UTC midnight as the daily VWAP anchor. UTC is a shared time reference rather than a local Indian trading clock. A position and its management counters can continue across midnight; the daily bias and setup count can reset without automatically liquidating the position.

The stock market's closing bell, daily square-off and overnight quarantine do not govern this continuous engine. Alternative VWAP anchors have not been optimized or enabled. A separate session-dependency audit found 116 references in the wider repository and zero active core stock-session occurrences.

10 How the scanner selects markets

The scanner checks every active market in the verified runtime `par` universe. It is direction-neutral: it can select a market that later produces a LONG or a SHORT setup. It does not create a buy signal merely because a market ranks highly.

For each market it obtains completed history, a valid book and asset context. It records ATR relative to price, spread, approximate traded notional, best-level depth, broader book depth, imbalance, EMA strength, VWAP distance, realized volatility, funding and available interest/volume data. The actual indicator values used for the decision are exposed in the signal records.

The selection process preserves the baseline normalized volatility, spread, history and Top 3 settings. Indian price bands and whole-share volume floors were replaced with comparable notional and depth measures, because a BTC quantity cannot be compared directly with a stock-share quantity or an FX quantity.

Scanner setting	Current example policy	Meaning
Minimum ATR fraction	0.004 or 0.40%	Market movement must reach the scanner floor
Maximum spread	15 basis points	Bid/ask difference must be small enough
Minimum history	300 completed bars	At least this much input is required
Top markets	At most 3	Creates a watchlist, not three simultaneous positions
Minimum window notional	0	No positive extra floor in the example; zero liquidity still fails
Minimum best-level notional	10	Requires positive executable depth on both sides

One basis point is 0.01%; 15 basis points equal 0.15%. The 0.40% scanner floor and the brain's 0.06% signal floor are different gates. Passing the lower signal floor does not bypass the scanner floor.

Qualified markets are ranked using equal-weight percentile comparisons of relative ATR, spread, window notional and best-level depth. A stable tie-break makes ordering reproducible. This is an infrastructure adaptation recorded in the migration notes, rather than an unreported change to VWAP entry rules.

If none qualify, the scanner returns NO_TRADE and an empty watchlist. The bot keeps the rejection evidence instead of lowering a threshold to produce activity.

11 Market specifications and order precision

`MarketSpec` describes a market's symbol, allowed quantity decimals, maximum leverage, asset ID and margin tiers. The client builds it from runtime metadata. This prevents a stock-era integer lot or two-decimal price assumption from being applied to a different contract.

The quantity step is `10` raised to the negative number of size decimals. For example, four size decimals mean a step of 0.0001. A requested size of 0.123456 becomes 0.1234; it is rounded down so the bot does not take more exposure than intended.

The price normalizer follows the implemented significant-figure and decimal-place rules. It respects the submitted limit: a buy limit rounds downward and a sell limit rounds upward when rounding is necessary. A value that cannot produce a positive valid size or price is rejected.

The minimum order value is validated at the end. A small calculated size is not automatically enlarged just to cross that minimum. Leverage is checked against the runtime market limit and, where supplied, the applicable notional tier. Changed metadata for an open position or order requires reconciliation rather than silently continuing.

12 Prices that must remain separate

The project distinguishes several prices because they answer different questions. Treating them as one number can create fictional fills or misleading P&L.

Price	Question it answers in this project
Best bid	What price is currently available to sell against?
Best ask	What price is currently available to buy against?
Mid	What is halfway between the best bid and ask?
Mark	What price does the local risk valuation use?
Oracle	What reference is used for paper funding and a separate equity estimate?
Last public trade	What was the last actual trade observation available to the feed?
Strategy reference	What VWAP level did the strategy use for its geometry?
Fill price	What price did the simulated or reported execution actually obtain?

`MarketSnapshot` records bid, ask, mid, mark, oracle, funding, open interest, side quantities, spread, accumulated side notional and imbalance. Last trade remains unknown when no actual observation is available. A mid or mark is not substituted for it.

Book imbalance compares the available notional on the two sides. A large imbalance can help explain a market condition, but this migration does not turn it into a new strategy signal. Separate clocks track the book and mark observation, so a fresh book cannot conceal an old valuation context.

13 Local order execution and partial fills

An `OrderRequest` carries the market, side, quantity, limit price, time in force, reduce-only flag, strategy identifier and optional client identifier. It is validated before a local order is created. The current request contract uses limit orders; immediate execution is represented with IOC behavior.

Order records move through states such as CREATED, SUBMITTED, ACKNOWLEDGED, OPEN, PARTIALLY_FILLED, FILLED, CANCEL_PENDING, CANCELLED and REJECTED. In PAPER mode these are local lifecycle states. A local SUBMITTED record is not evidence that an exchange received an order.

The paper engine simulates a taker fill only against a fresh two-sided BBO, meaning best bid and offer. A buy uses the observed ask; a sell uses the observed bid. Available displayed quantity limits the fill. The same displayed liquidity is consumed only once per observation, preventing repeated orders from reusing the same finite depth.

An IOC order fills whatever valid quantity is immediately available within its limit and cancels the remainder. A GTC order can remain open. A post-only ALO order is rejected if it would immediately cross; the simulator does not invent a maker fill when queue position and trade evidence are unknown.

For example, an order for 0.20 units facing only 0.15 eligible units can fill 0.15 and cancel the remaining 0.05. Accounting uses 0.15. Reduce-only orders can reduce or close a held position; they cannot open a new one, reverse direction or increase exposure.

Client order IDs support idempotence: repeating the same request returns its existing local order, while reusing that ID for a different request fails. Duplicate/conflicting fills are also checked. An offline response parser checks each individual exchange result; a top-level response marked successful can still contain a rejected operation.

14 Position management and exits

The migration reuses three verified baseline management methods: daily-bias updates, tick-level exit checks and completed-candle management. It compiles only those trusted, hash-checked functions from local source. The legacy module's broker constructors, old entry plumbing and stock accounting do not run through this adapter.

The initial stop and target retain their VWAP/ATR geometry. For a LONG, the stop is VWAP minus 2 ATR and the target is VWAP plus 4 ATR. For a SHORT, the directions reverse. Stock-specific two-decimal rounding was removed from the new risk body so small-price or small-ATR contracts keep meaningful geometry. Submitted order limits still pass exchange precision rules.

A target is monitored using the executable side of the current quote: bid for a LONG exit and ask for a SHORT exit. The regular strategy stop is evaluated on a confirmed completed candle close. This preserves the existing wick-immunity behavior, where an intraminute spike alone does not trigger that strategy stop.

After at least 15 held candles, the original rule can move the stop to the actual average entry price when the completed close is in profit. That is price breakeven; fees, funding or later slippage can still make the net result negative. A completed stop-loss exit starts the existing ten-observed-candle cooldown.

The current manager allows one tranche and the portfolio allows one global position. Older three-tranche constants remain frozen historical values; they do not mean pyramiding is enabled in the new engine. At most two new setups per UTC day are allowed by the example policy.

Before a direction flip, the new direction must pass the existing bias/cooldown and entry-permission checks. The old position must then be exited and reconciled before a replacement opens. A partial exit keeps its exit intent alive even if the next price moves away from the original trigger.

Separate paper capital-protection and margin-emergency checks can request a local reduce-only exit. They do not replace the strategy's normal stop. No real exchange protection order is active during migration.

15 Shared USDPR accounting

The paper account begins with hypothetical collateral of 20,000 USDPR in the example policy. This is a declared local starting balance. It is not a verified team-wallet balance or a claim about the organizers' deposit.

Unlike purchasing cash shares, opening a perpetual does not subtract its whole notional from cash as a stock purchase. The local model reserves margin and entry fees while recording the contract exposure. All positions share one collateral pool, although the current strategy permits only one global position.

The ledger stores signed fractional quantity, actual average entry, realized P&L, unrealized P&L, fees, funding, order reservations and margin estimates. Adding same-side fills updates the weighted entry price. A partial reduction realizes only the closed quantity. Repeated fill IDs cannot credit profit twice.

The main relationships are:

```
Long gross realized P&L = closed quantity x (exit fill - entry fill)
Short gross realized P&L = closed quantity x (entry fill - exit fill)
Cash = starting collateral + realized P&L - fees + funding cashflow
Mark equity = cash + unrealized P&L at observed mark prices
```

`initial_margin` describes capital assigned to exposure at the chosen leverage. `maintenance_margin_estimate` describes the model's minimum support using runtime tiers. `available_margin` considers equity, initial margin and open-order reservations. The model does not claim to reproduce every venue liquidation rule or an exact exchange liquidation price.

Open-position P&L remains unrealized. The engine does not invent an end-of-day close. If current marks are missing or stale, it refuses to publish current equity and writes known ledger facts with equity unset. The actual-account observer is maintained separately and never overwrites the paper cash balance.

16 Fees spread slippage and funding

The example fee schedule uses a maker rate of 0.0003 and taker rate of 0.0009, equivalent to 0.03% and 0.09%. The current paper fills are taker fills, so the simulator charges its explicitly declared taker

rate. Actual account accounting instead uses each reported fill's fee and fee token. Participant guide on venue mechanics

Spread is the difference between ask and bid. Buying at the ask already includes an entry-side spread effect. The cost model avoids adding that effect a second time. Slippage means an execution price differs from the reference used when planning; the current 5-basis-point allowance is an economic estimate, not an extra debit automatically subtracted from fill-based P&L.

Funding is a periodic cash transfer associated with perpetual exposure. The local model uses the signed position, observed oracle and hourly rate. At a positive rate a LONG pays; a SHORT receives. The paper formula is `cashflow = -signed quantity x oracle x hourly rate`. This can make cashflow positive or negative.

Paper funding is explicitly labeled PAPER_SNAPSHOT_ESTIMATE. It uses an observed hourly boundary snapshot. If a held position misses a boundary, if the observation is more than five seconds late, or if more than one hour was skipped, entries pause rather than creating a missing historical oracle or payment. A replay/review is then needed to resolve the accounting gap.

Actual account funding is different evidence. The observer reads `userFunding` and preserves the exchange-reported signed cash amount. It does not replace that payment with a rate-times-position estimate. The protocol's raw `usdc` field name is handled as the reported collateral amount for these HIP-3 rows, while the project retains USDPR labeling.

17 Position sizing and risk controls

Position sizing answers how much exposure the bot may take. It uses the smaller of the equity-based setup risk budget and the remaining daily allowance, then applies margin and displayed-depth limits. Quantity is rounded down after those limits. Leverage changes required margin; it is not permission to enlarge the loss budget.

At the example starting equity, 0.5% planned setup risk is 100 USDPR and the 1% daily budget is 200 USDPR. These are planned model limits. A market gap, incomplete exit or sudden liquidity change can produce a different realized loss, so the engine records actual fills rather than claiming the cap is a guaranteed outcome.

Six components separate the checks: PositionSizer limits quantity; ExposureManager limits concentration; LeverageManager validates market leverage; LiquidationGuard tests modeled margin headroom; FundingGuard limits expected funding cost; PortfolioRisk coordinates them with quote, spread, identity, setup-count and economic checks.

Current example limit	Value	Plain-language effect
Planned risk per setup	0.5% of current equity	Limits modeled loss per new setup
Daily loss allowance	1% of starting collateral	Pauses new setups when allowance is exhausted
Maximum drawdown	5%	Uses decline from the run's peak equity
Chosen leverage	1	Avoids increasing exposure through leverage in this policy
Maximum margin use	50% of equity	Keeps modeled collateral headroom

Current example limit	Value	Plain-language effect
Maintenance buffer	2 times estimated maintenance	Tests support after modeled stop loss
Gross exposure	At most 1 times equity	Caps total absolute notional
Symbol exposure	At most 1 times equity	Caps one market's notional
Net and directional exposure	At most 1 times equity each	Caps signed or one-way concentration
Book participation	10% of eligible top-side quantity	Avoids assuming the bot can take the whole displayed level
Mark versus oracle difference	At most 50 basis points	Rejects excessive valuation divergence
Expected funding cost	At most 0.1% of entry notional	Limits modeled carry cost

The economic assessment includes spread, explicit fees, estimated slippage and expected funding. Net profit must be positive. The preserved minimum net reward-to-risk is 1.2 in normal conditions and 0.8 below the existing low-volatility threshold of 0.1% ATR/reference. The unchanged scanner may reject those low-volatility markets before this later gate is reached.

These numbers are the checked-in example paper policy, not an optimized policy or verified account specification. They are available in `docs/paper_policy.example.json` so the team can inspect precisely what a run used.

18 Feed freshness and risk states

Freshness is checked separately for each market and each important observation. A fresh BTC order book does not prove its completed candle or mark is fresh. A stale GOLD book also does not automatically invalidate a healthy BTC candidate.

Observation or failure	Example limit
Tick and order-book age	5 seconds
Mark-context observation age	60 seconds
Completed-candle lag after close	120 seconds
Consecutive local order errors	3
Consecutive reconnect failures	5

The stream supervises connection health with pings and receive/pong deadlines. The implementation sends a ping around every 30 seconds, treats an extended receive gap of 45 seconds as unhealthy and checks a pending pong deadline of 10 seconds. These are the bot's local watchdog settings, not a statement that the exchange guarantees those response times.

CircuitBreakers exposes NORMAL, CAUTION, ENTRY_PAUSED and EMERGENCY. NORMAL indicates no current block. CAUTION can reflect a local market problem or shrinking financial

headroom. ENTRY_PAUSED forbids new entries because a global condition needs resolution. EMERGENCY reflects a severe equity/maintenance condition.

A state label does not itself submit an order or flatten an account. Separately coded paper-only management may request an exit when valid observations permit it. Repeated order errors can latch an entry block; an unrelated later success does not silently clear it. Empty or crossed books are isolated to their affected symbol instead of being treated as executable liquidity.

19 Reconnection and paper recovery

A dropped connection creates uncertainty. The engine immediately revokes feed readiness, reconnects, repeats subscriptions and restores REST history, context and books. It does not assume that the feed was complete during the interruption.

Each recovery has a generation number. The main thread accepts that generation only after market reconciliation and local-account reconciliation succeed. If the stream changes again while recovery runs, the old generation cannot authorize entry. This prevents a recovery result from an earlier connection from being applied to a newer one.

Paper reconciliation replays known local fills and explicit funding records. It compares the reconstructed cash and quantities with the current local objects. Where the evidence supports a repair, it restores actual average entry and quantity while preserving virtual stop/target state. Missing fill history or missing management evidence pauses entry.

An idle market gap is handled differently from a held-position gap. With no exposure, the cursor can resume at a known completed bar while keeping the existing cooldown state and counting only observed bars. With an open position, missing management bars require review because a stop or another state change may have occurred during the gap. No missing bars are invented to advance counters.

Continuous operation and reconnect recovery are implemented, but an interrupted process is not automatically resumed from a saved report. A fresh output directory is required for a new run. A prolonged exposed paper session and a complete operational restart exercise remain validation work.

20 Read only exchange account truth

`HyperliquidAccount` accepts a public team address and exposes `get_balance`, `get_equity`, `get_positions`, `get_open_orders`, `get_fills` and `get_funding_history`. It also retains `get_position` and `get_funding` helpers. Account reads use the team address, not an API wallet's address.

The observer validates the address format and rejects the all-zero placeholder. Its balance method returns reported USDPR collateral; it does not call that number free margin. Equity reads require the reported account abstraction to be unifiedAccount and avoid double-counting overlapping account balances.

Actual fills preserve reported price, fractional size, fee, fee token, side, starting position and closed P&L. Funding preserves actual reported signed cashflow. Composite identifiers support deduplication; conflicting duplicates and invalid or future timestamps fail. A full-size history page is treated as potentially truncated and requires bounded backfill rather than an unsupported completeness claim.

Account recovery reads positions/orders before and after the event window to check stability. It adopts the observed exchange quantity and open-order view locally; it does not send a repair order to

the exchange. Missing fills, unexplained quantity changes, untracked orders or missing P&L evidence keep entries paused.

Recurring recovery uses an inclusive overlap around the previously successful window. The overlap captures recent events while duplicate identities prevent double accounting. An unresolved gap causes the original window to be revisited until evidence explains it. If required P&L fields are missing, the result stays unknown instead of becoming zero or estimated profit.

These account branches have been tested with fixtures. No Team Anchorium public address was supplied for a real-account integration run, so the team's actual funding, permissions, fees, positions and recovery remain unverified.

21 Event timing and complete candidate traces

The new event records contain `exchange_ts`, `received_ts` and `processed_ts`. These answer when the external observation occurred, when it arrived locally and when the bot processed it. They use explicit UTC values where those clocks are available.

An exchange timestamp remains null for a local decision, simulated fill, estimated funding record or timestamp-free API context. The code does not turn the local receipt time into a fictional exchange publication time. Source clocks for real books, completed candles, trades, account fills and funding are retained when available.

The timing helper reports exchange-to-receive age, receive-to-process duration and clock-skew detection. An old historical candle can have a large age even when the HTTP request was fast; that age is not proven network latency. A local fill's submit-to-fill duration measures the simulation. Actual order-ack and exchange-fill latency cannot be established while signed execution is disabled.

Every scanned candidate gets a candidate ID. The trace connects MARKET_DATA, SIGNAL, RISK_DECISION, EXECUTION_DECISION, ORDER, FILL, POSITION_AND_PNL, EXIT_FILL and TRADE_PNL. A rejected candidate stops at its real stopping point. It does not receive an invented downstream order or profit.

Signals include the frozen VWAP, EMA9, EMA21, ATR, direction and setup output. Risk records include quantity, stop, target, estimated economics and the rejection reason when applicable. Orders and fills preserve the original candidate link, and later exits retain the opening position's candidate ID even between scans. This makes a particular trade or rejection explainable from its recorded evidence.

22 Results files and session identity

Each paper or replay session has a `results/` directory. JSONL means one JSON object per line, which makes event files appendable and machine-readable. CSV is a table that can be opened in spreadsheet software. Empty event files and header-only CSV files accurately represent sessions with no relevant activity.

File	What the team can learn from it
orders.jsonl	Local requests and order lifecycle states
fills.jsonl	Simulated executed prices, sizes, fees and timing

File	What the team can learn from it
signals.jsonl	Indicator values, direction, selection and signal status
rejected_signals.jsonl	Why a candidate was prevented from trading
equity.csv	Local cash, mark equity, oracle estimate and margin
positions.csv	Held quantities, entry prices or an explicit flat snapshot
funding.csv	Recorded local funding and its source label
fees.csv	Local fees linked to fills and orders
daily_metrics.csv	Last observed daily values and provisional return metrics
health_events.jsonl	Risk transitions, stream recovery and other health events
candidate_traces.jsonl	Connected decision and execution history
session_manifest.json	Code, configuration, strategy and parameter identity

The original report, event, equity and candle files are also retained alongside `results/`. A bounded public run without an account observer produces a replay tape. Reports are refreshed while event exports use cursors to avoid repeating the same events. Repeated saving at the same logical equity timestamp does not duplicate that snapshot.

The manifest stores the start time, Git commit, branch, dirty status, working-source hash, configuration hash, strategy identity/check, adapter hash, risk hash and full parameter set. Dirty means working files differ from committed code. Recording both the commit and source hash prevents the commit from being mistaken for the entire executed version.

The software uses real content hashes rather than inventing version labels such as VWAP-v4 or HL-v1.3. Raw candles and source evidence remain linked to the run. A halted session flushes known facts without fabricating a current valuation.

23 Paper metrics and competition scoring

The local report records return on hypothetical starting collateral, realized and unrealized P&L, maximum observed drawdown, closed-trade count and win rate when enough closed trades exist. With no closed trades, win rate stays unknown. Open profits are not treated as closed wins.

Gross exposure sums absolute position notional. Net exposure retains direction. The reports also show LONG and SHORT notional, per-symbol exposure, directional concentration, position count and margin utilization. The current one-position rule limits portfolio complexity; correlation modeling across a larger simultaneous portfolio has not been implemented.

`equity.csv` separates mark-valued paper equity from `oracle_equity_estimate`. `daily_metrics.csv` uses the last observed sample of each UTC day. Today's row is PROVISIONAL. A daily return requires an adjacent previous day, missing days are not interpolated, and sample volatility stays empty until there are at least two observed daily returns.

These exported samples are useful for analysis but can be sparse or cover only part of a day. They are not the organizer's official equity series, cross-team normalized score or jury score. The latest

public probe's unchanged 20,000 USDPR and zero P&L describe a flat hypothetical account during a no-trade observation.

24 Replay and the blocked live mode

Replay lets the team ask whether the same captured input produces the same decisions again. The recorder saves public REST requests/responses and the logical clock values seen by the bot. The tape includes historical completed bars and observed quotes rather than candles alone.

The replay runner verifies the checksum, loads the recorded policy, validates the same market metadata and uses the same frozen strategy, normalization, BBO-fill model and portfolio accounting. It constructs no HTTP session, socket or signer. Local replay order/fill identifiers are deterministic.

Replay requires exact chronological operation consumption. It rejects tampering, a different request, a missing operation, a backward clock and unused operations. It never searches forward to find a favorable price. A new adapter/clock protocol can make an old tape incompatible, which is why the session revision and hashes matter.

The current runner supports one complete bounded REST scan. It does not provide a multi-month backtester, continuous-WebSocket replay or automatic restart recovery. Candles alone cannot establish historical spreads, available depth, mark context or intraminute fill order, so the runner does not invent them.

REPLAY describes the offline run context while the environment's safety configuration stays PAPER and false. Setting `EXECUTION_MODE=REPLAY` or LIVE as a routing override is rejected by the migration policy. The LIVE executor exposes place, submit, cancel, modify, close and leverage methods, but every signed-action boundary raises LiveTradingDisabled before transport access.

25 Wallet readiness and secret protection

The team can develop and test public market data without a wallet secret. Blank or zero wallet addresses display NOT_CONFIGURED. A nonzero address of the correct shape displays CONFIGURED_UNVERIFIED. A correctly formatted address alone does not prove funding, permissions or account readiness.

The observed default readiness is wallet ready NO, market-data ready YES, paper ready YES, live ready NO and signed actions DISABLED. This avoids presenting the all-zero placeholder as a funded account.

Both `.env.example` files contain placeholders:

```
HL_NETWORK=testnet
HL_DEX=par
TEAM_WALLET_ADDRESS=
API_WALLET_PRIVATE_KEY=
EXECUTION_MODE=PAPER
ALLOW_LIVE_ORDERS=false
```

The private-key placeholder is not a request to insert a key during migration. The team-wallet seed and private key remain outside the repository. The ignore rules cover `.env`, other environment files, `*.key`, `wallets/`, `secrets/` and runtime results, while keeping `.env.example` trackable.

The public client permits an explicit allowlist of `/info` reads and rejects signed-action parameters. Wrong network, wrong DEX and contradictory safety settings fail. Angel One remains permanently disabled in this clone. No automatic activation path, team-wallet approval or account-administration mutation has been implemented here.

26 Testing and the observed public results

The test records show progressively broader engineering coverage. They include numerical checks, controlled exchange fixtures, strategy comparisons, failure cases and orchestration tests. Fixtures deliberately create scenarios such as partial fills or conflicting history; they do not establish that those events occurred in the Team Anchorium account.

Checkpoint	Passing tests	Main scope
Inherited version 6	244	Execution safety and fill/account recovery
Stock discovery	303	Staged universe discovery
Discovery repair	338	Provider/discovery correctness
Runtime repair	382	History, startup, pacing and observation
Migration phases zero through five	499	Isolation, frozen strategy and par data
Policy reinforcement	527	Production lockout tests
Integrated crypto paper infrastructure	595	Paper execution, risk, ledger and management
Recovery and risk controls	645	Account truth, freshness and reconnect barriers
Audit and replay	678	Timing, results, readiness and deterministic replay

The latest saved 678-test run has zero failures, errors and skipped tests. Its new audit/replay group contains 33 cases; the preceding recovery group added 50, and the earlier paper group contained 68 focused checks. These groups are subsets of the relevant full suites, not separate totals to add to 678. The full suite was not rerun solely to write this report; its saved machine-readable record was checked and the freeze/Phase 0 checks were rerun.

Public network observations are separate evidence:

Observation on 2 October 2026	Recorded result
Initial market-data check	10 markets; 58 completed BTC candles in a requested 60-minute window; 10 stream messages; zero orders
Paper probe at 07:52 UTC	10 markets; NO_TRADE; six stale histories, one insufficient history, remaining markets failed baseline ATR/spread checks
Continuous probe at 08:23 UTC	Two scans of ten markets; two equity snapshots; NO_TRADE; zero orders

Observation on 2 October 2026	Recorded result
Recovery probe at 09:18 UTC	180 stream data messages; accepted local recovery generation; NO_TRADE; zero orders
Audit probe at 12:10 UTC	10 markets, all stale completed candles; 10 signal/rejection records and 30 trace events; zero orders/fills
Offline replay of the audit probe	6,969 recorded operations consumed; diagnostics, decisions and portfolio matched exactly; no network or signed actions

Some probes preceded later refinements, as recorded in their validation notes. They are dated snapshots, not a continuous record of current market quality. The 09:18 recovery run had a flat portfolio and zero reconnects, so it observed startup recovery; forced reconnect failures and exposed-position cases were established by offline tests.

Zero trades do not by themselves prove either failure or a profitable strategy. In these recorded windows, the filters and freshness checks correctly prevented entry. A profitable full session, meaningful trade sample, funded-account cycle and prolonged exposed paper run have not yet been demonstrated.

27 What is implemented and what remains

The current body provides public market connectivity, runtime specifications, compatible candle input, preserved strategy management, scanning, paper lifecycle/accounting, perpetual risk checks, freshness supervision, local recovery, account-observer interfaces, audit results and bounded replay.

The following work remains material before autonomous competition execution:

1. Verify the funded Team Anchorium address, permissions, unified account mode and actual collateral through controlled account integration.
2. Implement and review the signed testnet transport after the user's explicit authorization; changing a flag cannot supply this missing implementation.
3. Verify actual fees, reported funding, partial fills, cancellation and a full entry-to-flat account cycle.
4. Run longer paper observation with healthy data and exposure, including hourly funding boundaries, interruptions and stop/target management.
5. Complete operational restart/resume design and verify how management state is reconstructed across process interruption.
6. Expand historical evaluation using complete contemporaneous quotes/context and held-out data if broader replay or strategy research is required.
7. Add maker-queue modeling, exact venue liquidation modeling, multi-position correlation controls or a new dashboard only as separate scoped work.
8. Assess genuine strategy performance before optimization, then consider separately authorized AI/ML changes with versioned evaluation.

No meaningful completion percentage can be inferred from the number of files or tests. Account readiness, exchange execution correctness and measured trading performance are separate

milestones. The present report supports a clear claim of substantial tested paper infrastructure while identifying those remaining milestones.

28 The controlled practice sequence

The organizer guide places account setup after funding. The team wallet approves an API wallet, then account mode, agent approval and collateral are verified. Future signed trading uses that separately approved API wallet. This sequence has not been performed by the migration. Official account setup

After funding and explicit user authorization, the review sequence is: verify the public account and collateral; manage an API wallet outside source/shared logs; approve it; configure unified mode; verify permissions with read-only queries; review the signed adapter; run a tiny controlled entry; confirm its actual fill; exit reduce-only; confirm flat; reconcile fees and P&L. LONG, SHORT, cancellation, rejection, partial fill, reconnect and recovery cases must then be exercised before autonomous competition execution.

The official schedule currently lists practice starting on 5 October 2026, scored trading on 19 October, trading ending on 6 November and report submission on 9 November. Published schedule

The initial practice target is correct market, symbol, side and quantity; no duplicate order; no unexplained position; no fictional P&L; no entry on stale data; reliable recovery; and bounded leverage. The team should review the evidence against these conditions before increasing autonomy. Strategy optimization and AI/ML experiments should follow as separately versioned phases.

29 Operating the current project

Run the following commands from the cloned project root. The existing `.venv` is the development environment. A new setup installs the new core requirements in its own virtual environment; legacy regression requirements are listed separately. Installing legacy dependencies is not needed to obtain broker credentials for the new public-data path.

Verify preservation and safety. These checks require no wallet secret:

```
.venv/bin/python scripts/check_strategy_freeze.py
.venv/bin/python scripts/phase0_check.py
```

Inspect public data. This command observes prices, completed candles and a short public stream. It places no orders:

```
.venv/bin/python scripts/par_market_data_check.py \
  --symbol par:BTC --bars 60 --signals --stream-seconds 8
```

Run one bounded paper scan. Review `docs/paper_policy.example.json` first. The runner prints the new output directory and creates a replay tape when no account observer is configured:

```
.venv/bin/python scripts/run_crypto_paper.py --cycles 1
```

Observe continuously. The example scan cadence is 300 seconds; stream-based management continues between scans. Ctrl-C ends the run:

```
.venv/bin/python scripts/run_crypto_paper.py \
  --cycles 0 --scan-interval-seconds 300
```

Inspect evidence. Open `report.json`, the session's `results/` directory and its candle records. Check readiness, stale-data reasons, watchlist, risk state, orders, fills and balance origin. A session returning NO_TRADE may be the correct outcome.

Replay a captured session. Replace the two paths with the actual tape and a fresh output directory:

```
.venv/bin/python scripts/run_crypto_replay.py \
  --tape '<session>/replay_tape.json' \
  --output '<new-replay-session>'
```

Run regression checks when needed. Changed source or a failed check can justify rerunning the suite:

```
.venv/bin/python -m pytest -q algo-trading-bot
.venv/bin/python scripts/inventory_session_dependencies.py
.venv/bin/python scripts/inventory_legacy_dependencies.py
```

An optional account observer requires an explicitly supplied public team address and an explicit history-start timestamp. It remains read-only and does not enable trading. Until the team is ready for that separately controlled step, use the default public paper run without an address.

30 Code and file reference

Paths in this section are relative to the project root. The tables identify the main active implementation and inherited references so a reader can connect the explanation to the code.

Core strategy and orchestration

File in algo-trading-bot	Role
signal_brain.py	Frozen indicators and LONG/SHORT signal decisions
strategy_freeze.py	Checks the baseline manifest
frozen_strategy.py	Feeds normalized candles into the unchanged brain
candle_normalizer.py	Validates and converts completed OHLCV input
session_manager.py	UTC daily anchor and continuous-session behavior
strategy_contract.py	Loads protected parameters and verified management methods
crypto_strategy_state.py	Connects baseline management to new venue callbacks
market_regime_scanner.py	All-par qualification, ranking and diagnostics
market_stream.py	Feed supervision and recovery acceptance
crypto_paper_bot.py	Coordinates signal, risk, execution, ledger and management
venue_config.py	Enforces the migration configuration
order_guard.py	Blocks real exchange mutations
session_results.py	Writes audit exports and session identity

Exchange package

File in algo-trading-bot/exchange	Role
base_exchange.py	Shared market, price, candle, fill and account contracts
hyperliquid_client.py	Testnet public reads and mandatory par discovery
hyperliquid_market_data.py	REST parsing, public streams and gap repair
market_snapshot.py	Distinct execution, valuation and depth observations
order_lifecycle.py	Request validation, states and offline response decoding
hyperliquid_execution.py	Guarded future LIVE interface
paper_execution.py	Local orders, depth consumption and BBO fills
paper_portfolio.py	Shared fractional USDPR accounting and margin
perpetual_economics.py	Fee and expected trade-cost calculations
paper_account.py	Read-only view of local paper facts
paper_reconciliation.py	Reconstructs and compares local ledger/state evidence
hyperliquid_account.py	Public-address actual account reads
account_events.py	Exchange-reported signed funding records
hyperliquid_reconciliation.py	Actual account truth and incremental event recovery
event_timing.py	UTC timestamps, durations and skew flags
replay.py	Recorded public input and deterministic local execution
readiness.py	Honest wallet, market and execution readiness

Risk package

File in algo-trading-bot/risk	Role
models.py	Explicit policy fields and risk-decision records
position_sizer.py	Quantity constrained by risk, margin and depth
exposure_manager.py	Gross, net, directional and symbol exposure
leverage_manager.py	Runtime leverage validation
liquidation_guard.py	Modeled margin and maintenance headroom
funding_guard.py	Expected carry-cost restriction
portfolio_risk.py	Coordinates the admission checks and economics
circuit_breakers.py	Financial, global and symbol entry states

Operating scripts and inherited references

File or group	Role and current standing
scripts/run_crypto_paper.py	Bounded or continuous current paper runner
scripts/run_crypto_replay.py	Current bounded offline replay runner
scripts/par_market_data_check.py	Public connectivity and strategy-input probe
scripts/check_strategy_freeze.py	Preservation verification
scripts/phase0_check.py	Isolation and mutation-lock checks
scripts/inventory_legacy_dependencies.py	Classified legacy-reference report
scripts/inventory_session_dependencies.py	Stock-session reference and active-core audit
safe_trader.py and safety_journal.py	Inherited stock safety/accounting references
instrument_registry.py and market_ discovery.py	Inherited NSE identity/discovery references
history_service.py and session_bootstrap.py	Inherited stock history/pacing/repair references
replay_engine.py and research_harness.py	Inherited stock research tools; distinct from current crypto replay
libquant_engine	Inherited C++ parser, buffer, VWAP and Python bindings

31 Glossary for the team

Term	Easy explanation
Autonomous	Decisions follow code without someone manually placing each entry
API	A structured way for software to request data or actions
REST	Request/response communication used for snapshots and history
WebSocket	A connection that can continuously deliver updates
DEX	The selected exchange group; this project requires par
Testnet	A test network using non-monetary test assets
Perpetual	A futures-style contract without a fixed expiry
Collateral	The account balance supporting contract exposure
USDPR	The competition collateral/accounting unit
LONG	Exposure that benefits from a price rise
SHORT	Exposure that benefits from a price fall
Notional	Quantity multiplied by the relevant price

Term	Easy explanation
Margin	Collateral assigned or required to support exposure
Leverage	Exposure relative to the assigned margin
Liquidation	Venue risk action when required support is inadequate
Funding	Periodic signed cashflow associated with perpetual positions
Candle	Open, high, low, close and volume for an interval
VWAP	Running volume-weighted average price
EMA	A recent-price-weighted moving average
ATR	A measure of typical bar movement
BBO	Best bid and offer, the top executable quote levels
Spread	Ask minus bid
Basis point	One hundredth of a percentage point
IOC	Fill immediately within the limit and cancel the remainder
GTC	Keep an unfilled order open until cancelled or completed
ALO	Post-only order intended to add liquidity
Reduce-only	An order that may shrink existing exposure only
Fill	Quantity that actually executed in the selected model or account
Reconciliation	Comparing recorded state with authoritative evidence
Idempotent	Repeating an operation does not apply its effect twice
Drawdown	Decline from an earlier peak equity
Hash	A fingerprint of file content or configuration
Provenance	A record of where a result and its inputs came from
Fixture	Controlled test input used to exercise a particular case

32 Example of one complete paper trade

This arithmetic example uses fictional prices and a fictional quote sequence for teaching. It is not an observed BTC trade. Quantity is fixed at 0.5 units here only to make the arithmetic clear; the actual bot calculates its quantity through the risk engine.

Suppose the frozen brain produces a LONG candidate with VWAP 100 and ATR 2. The geometry gives a virtual stop of 96 and target of 108. The scanner, state permissions, freshness and risk checks would all need to pass before an order could exist.

Assume an eligible ask of 100.01 with sufficient displayed depth. The local buy fill is 100.01, rather than the VWAP reference of 100. At the example taker rate, the entry fee is `0.5 x 100.01 x 0.0009 = 0.0450045` USDPR.

If one eligible hourly funding boundary occurs with oracle 100 and rate 0.0001, the LONG's estimated cashflow is `-0.5 x 100 x 0.0001 = -0.005` USDPR. That payment is labeled a paper estimate and requires an observed timely boundary.

Later suppose the valid exit bid is 108.10. The target condition is reached and a reduce-only sell fills 0.5 units at 108.10. The exit fee is `0.5 x 108.10 x 0.0009 = 0.048645` USDPR. Gross fill-based profit is `0.5 x (108.10 - 100.01) = 4.045` USDPR.

The example's net setup result is `4.045 - 0.0450045 - 0.048645 - 0.005 = 3.9463505` USDPR. The ledger returns to zero quantity, releases margin and records the completed setup. Estimated slippage is not debited a second time after actual simulated fill prices are already used.

The trace would connect the source candles, indicator values, risk decision, normalized request, entry fill, fee, funding, target observation, reduce-only exit, flat position and net P&L. If only part of the exit quantity filled, the ledger would realize only that part, retain the remaining position and continue the exit intent.

33 Evidence and further reading

The local evidence below preserves the implementation history and the detailed validation behind this report. Older checkpoint limitations should be read together with the later guides, because later work implemented some previously pending components.

Local reference	What it supports
docs/implementation_v6_report.md	Inherited execution-safety architecture and 244-test checkpoint
docs/discovery_validation_2026-10-01.json	303-test discovery checkpoint
docs/discovery_repair_validation_2026-10-01.json	338-test discovery repair and read-only validation
docs/runtime_repair_implementation_2026-10-01.md	382-test runtime repair and historical research limits
docs/strategy_freeze.json	Exact protected file, methods, parameters and baseline
docs/infrastructure_adaptations.json	Narrowly documented market/session/precision adaptations
docs/phase1_5_validation_2026-10-02.json	First crypto checkpoint, public data and 499 tests
docs/migration_policy_validation_2026-10-02.json	527-test policy checkpoint and mutation lockout
docs/crypto_paper_migration_2026-10-02.md	Integrated paper architecture and operating explanation
docs/crypto_paper_validation_2026-10-02.json	595-test paper checkpoint and assumptions

Local reference	What it supports
docs/crypto_recovery_migration_2026-10-02.md	Actual-fill truth, feed recovery and risk states
docs/crypto_recovery_validation_2026-10-02.json	645-test recovery checkpoint and public probe
docs/crypto_audit_replay_2026-10-02.md	Timing, outputs, replay and wallet/practice run guide
docs/crypto_audit_validation_2026-10-02.json	Latest 678-test, public audit and matching replay evidence
docs/crypto_audit_tests_2026-10-02.xml	Machine-readable latest suite results
docs/paper_policy.example.json	Exact example collateral, costs, scanner and risk limits
docs/crypto_public_audit_probe_2026-10-02.json	Saved genuine public audit report with zero trades

The external competition references were checked on 2 October 2026: overview and evaluation, participant guide and published schedule. The venue's rules and market specifications should be rechecked before future funded-account integration.